

BruitTrack

Rapport d'audit detaille du code, de l'exploitation et de la conformite aux exigences

Date de l'audit	23 aout 2026
Portee	Code Python, configuration, tests, packaging, service systemd et documentation du depot.
Revision auditee	Git 600a0be - arbre de travail avec le dossier non suivi files/ preserve et non modifie.
Verdict	Base logicielle solide et testee hors materiel, mais non prete a revendiquer une conformite acoustique-legale ou un deploiement autonome sans corrections prioritaires.

Conclusion executive

Le projet presente une architecture claire et adaptee a un PC basse consommation : traitement stereo borne, SQLite WAL, ecritures groupees, UI HTTP sans dependance lourde et tests deterministes. Les controles locaux passent : 92 tests pytest, ruff check et le harnais hors materiel 5/5.

Toutefois, deux assertions centrales ne sont pas demontrees par le code. D'une part, la regle de filtrage legal compare une emergence spectrale relative au plancher a un seuil reglementaire dB(A), sans calibration ni niveau residuel. D'autre part, l'installation normale installe le paquet sans l'extra audio, alors que sounddevice est indispensable au daemon. Ces deux points dominent le risque de production.

1. Methode, preuves et limites

Audit en lecture seule : inspection des modules src/bruittrack, des tests, de config.toml.example, pyproject.toml, tools/ et du service systemd. Aucune correction applicative n'a ete apportee. Les constats de code sont references par chemin et ligne approximative de la revision auditee.

Controles executes

Controle	Resultat	Interpretation
python -m pytest -q	92 passes en 13,95 s	Bonne couverture unitaire et API sans carte son.
python -m ruff check .	Passe	Aucune erreur de lint signalee.
python -m ruff format --check .	Echec : 2 fichiers	tests/test_viz_api.py et tools/_purge_0hz.py doivent etre reformates.
python tools/module_check.py --offline	5/5 OK	CLI, config, fingerprint, store et API verifiees hors materiel.
pip check / imports	pip check OK; sounddevice absent	Le chemin capture reel reste non executable dans l'environnement installe avec .[dev].

Limites

- Aucun microphone M-Track Plus ni HP T620 n'etait disponible : latence ALSA, stabilite 24/7 et budget CPU/RAM ne sont pas re-certifies ici.
- Un audit de code ne remplace pas une metrologie acoustique : les niveaux dB(A), l'etalonage du capteur et le protocole de mesure doivent etre verifies sur site.
- La securite reseau est evaluee selon le code expose ; aucun scan externe n'a ete realise.

2. Architecture evaluee

Le flux respecte globalement la separation attendue : capture -> DSP -> floor -> detection/fingerprint -> SQLite -> visualization. Les responsabilites sont bien isolees dans capture.py, dsp.py, events.py, store.py, pipeline.py et viz.py.

Couche	Points solides	Point de vigilance
Capture	Import differe de sounddevice, queue bornee a 100 blocs, abandon du plus ancien bloc en surcharge.	Dependance audio non installee par le flux normal; perte de blocs seulement signalee, sans compteur persiste.
DSP	SOS SciPy, decimation exacte configurable, Welch/Hann/EMA, floor median et etat borne.	La borne inferieure 2 Hz et freq_max 150 Hz exigees ne sont pas implementees.
Detection	Hysteresis, debounce, decoupage 30 s, fingerprint de 16 octets, cluster en memoire.	La comparaison 'both' ne respecte pas le voisinage de plus ou moins 2 bins annonce.
Stockage	WAL, synchronous=NORMAL, lots et index t0/cluster. Les lectures ouvrent leur propre connexion.	Les exemplaires retires par retention ne sont nettoyes que via une commande manuelle.
Visualization	Serveur stdlib, API simple, endpoints testes, WAV genere a la demande.	Ecoute 0.0.0.0 et CORS '*' sans authentication ni limite de requete.

Qualite de conception

La modularite et le test hors materiel sont les atouts majeurs. La gestion de la base est raisonnable pour l'objectif : le tampon d'ecriture est protege par RLock, l'echec de flush conserve le lot et les connexions fichier sont courtes. Le fallback DSP pur Python est utile pour la portabilite, mais ne doit pas etre considere compatible avec le budget CPU sans SciPy.

3. Synthèse des constats

Severite : Critique = empeche une promesse centrale; Haute = risque operationnel ou divergence majeure de specification; Moyenne = robustesse, maintien ou securite; Basse = hygiene.

ID	Severite	Constat	Impact
F-01	CRITIQUE	Le flag legal compare des dB spectraux relatifs au floor a une limite dB(A).	La selection 'seulement les evenements illegaux' n'est pas defensible legalement.
F-02	HAUTE	La duree cumulee et la regle de mesure 10 s ne sont pas appliquees au pipeline.	Les seuils legaux et le statut de validite peuvent etre faux.
F-03	HAUTE	pip install -e . et .[dev] n'installent pas sounddevice; install_hp.sh n'installe pas .[audio].	Le service de capture peut echouer immediatement apres installation.
F-04	HAUTE	min_event_hz=2 et freq_max=150 demandes sont absents; le default est 48 Hz.	Les 1 Hz restent detectables et les evenements 48-150 Hz ne sont pas analyses.
F-05	MOYENNE	Viz est publique sur toutes interfaces avec CORS permissif et aucune authentication.	Triage et metadonnees modifiables par tout client joignable.
F-06	MOYENNE	Validation config incomplete et device vide accepte.	Derive possible du materiel cible et capture sur le peripherique par default.
F-07	MOYENNE	Retention supprime les events, pas les exemplaires automatiquement.	Occupation disque residuelle et donnees sensibles potentiellement conservees.
F-08	BASSE	Deux fichiers ne respectent pas le format Ruff et des caracteres mal encodes apparaissent dans des commentaires/docstrings.	Dette de maintenance et documentation terminal degradee.

4. Constats critiques et hauts

F-01 - Confusion entre detection DSP et conformite acoustique

events.py:339-347 calcule le bit FLAG_OVER_LEGAL depuis max(peak_lvl_g, peak_lvl_d). Ces valeurs sont l'emergence par bin de PSD lisse par rapport a la mediane interne (pipeline.py:123-136; dsp.py:313-324), pas bruit_ambient dB(A) moins bruit_residuel dB(A). Aucun etalonnage de chaine micro, ponderation A, integration de niveau, acquisition de bruit residuel ni incertitude de mesure n'est modelise. legal.py implemente correctement la formule abstraite, mais le pipeline ne lui fournit pas ses entrees physiques.

Recommandation : separer explicitement 'detection technique' et 'constat legal'. Ne poser un flag legal qu'a partir de mesures etalonnees LAeq/ponderation A, d'un residuel documente et d'un protocole de mesure trace. Tant que cela n'existe pas, renommer le flag en 'depasse_seuil_detection' et retirer toute promesse de conformite.

F-02 - Duree cumulee et validite de mesure non suivies

La limite est calculee avec duration_s de chaque segment (events.py:345), alors que la regle exige la duree cumulee de la nuisance. Un evenement continu est decoupe a 30 s, ce qui reinitialise implicitement le correctif a +6 dB. La condition d'invalidite de legal.py:66 est testee unitairement, mais n'est jamais appelee par EventDetector ni persistee. L'extrait audio ne fait que 256 ms et la capture de validite de 10 s n'est pas materialisee.

Recommandation : relier les segments contigus a une session de nuisance, conserver sa duree cumulee et son historique de mesure; enregistrer les champs de validite et de provenance. Une courte occurrence doit avoir une fenetre de mesure ambiante documentee d'au moins 10 s.

F-03 - Installation incompletement reproductible

pyproject.toml declare sounddevice dans l'extra audio, tandis que la CI installe .[dev] et tools/install_hp.sh lance pip install -e . seulement. Le test d'import execute pendant l'audit echoue avec ModuleNotFoundError: sounddevice. Le harnais offline ne couvre logiquement pas ce chemin. Le service systemd lance pourtant start, donc AudioCapture.start echouera sur une installation propre.

Recommandation : installer .[audio] dans le script HP et tester le paquet avec les extras de production. Ajouter une preflight bloquante qui verifie import sounddevice et l'ouverture du device configure.

5. Specification, securite et exploitation

F-04 - Bande de detection non conforme a la specification courante

AGENTS.md demande `min_event_hz=2.0` et `freq_max=150.0`, avec exclusion des bins sous 2 Hz. Pourtant `DspConfig/config.toml.example` restent a `freq_max=48.0` (`config.py:33,172`) et `EventDetector` recherche tous les bins a partir de 1 (`events.py:250-258`). Aucune validation ou test ne porte `min_event_hz`. Le plan de travail `IMPROVEMENTS.md` marque explicitement ces travaux comme ouverts.

Recommandation : ajouter `min_event_hz` a `DspConfig`, valider $2 \leq \text{min_event_hz} < \text{freq_max} \leq 150$, masker les bins avant detection et enrichir les tests : 1 Hz refuse, 120 Hz detecte. Synchroniser `AGENTS.md`, `decision-log`, `README` et l'exemple TOML.

F-05 - Surface HTTP trop ouverte par default

`config.toml.example` ecoute sur 0.0.0.0 et `viz.py` renvoie `Access-Control-Allow-Origin: *`. Le POST `/api/clusters//triage` accepte flags et label sans authentication ni taille maximale de corps. Pour un outil strictement local, cela est inutilement expose. La contrainte de projet exclut le reseau/cloud, ce qui rend le choix encore moins coherent.

Recommandation : default 127.0.0.1, CORS desactive, et exposition explicite uniquement derriere un proxy local authentifie si elle devient necessaire. Borner limit sur `/api/events` et `Content-Length` sur POST.

F-06 et F-07 - Garde-fous operationnels

- `Config.validate` n'impose ni `sample_rate=48000`, ni `decimation=48`, ni `fs_low=1000`, ni un device configure. Elle n'impose pas non plus `floor_history_len >= 1` ou `noverlap >= 0`. Ces invariants doivent etre verifies avant l'ouverture du flux.
- `apply_retention` supprime les lignes events, mais ne supprime pas l'exemplaire du cluster devenu absent. La commande `prune` existe, mais n'est ni appelee periodiquement ni couplee a la retention. Prevoir une operation atomique ou un nettoyage quotidien.
- Les preuves de performance historiques sont utiles, mais aucun test CI ne mesure CPU/RSS sur le T620. Garder la commande `perf` et ajouter une procedure de recette hardware versionnee.

6. Points positifs et couverture de tests

Ce qui est bien realise

- Le pipeline manipule les deux canaux a chaque tick et le filtre conserve correctement son etat entre blocs. La voie SciPy est une decision pragmatique pour le budget CPU.
- Le floor tracker est borne en memoire et amorce son historique, ce qui evite un faux pic au demarrage. La detection a hystereses, debounce et limite de duree est lisible.
- SQLite est configure en WAL/NORMAL, les ecritures sont groupees et le lot n'est efface qu'apres succes. Les API de lecture utilisent des connexions isolees.
- Les tests couvrent DSP, evenement, store, API, config, capture simulee et regles legales unitaires. Ils sont deterministes et ne dependent pas de materiel.

Lacunes de test prioritaires

Domaine	Test a ajouter
Installation	Dans un venv vierge : lancer tools/install_hp.sh ou l'equivalent non-root, verifier import sounddevice et la commande devices.
Legislation	Tests d'integration refusant le flag legal sans valeurs dB(A) et calibration, puis scenarios de duree cumulee multi-segments et fenetre 10 s.
Bande	1 Hz ne produit aucun evenement; 120 Hz le produit avec freq_max=150; validation des bornes TOML.
Reseau	Le serveur par defaut est boucle locale; les POST non autorises et corps excessifs sont rejetes.
Materiel	Recette T620 10 min avec M-Track : perdus de queue, CPU, RSS, journal systemd et arret propre.

7. Plan de remediations recommande

Priorite	Action	Critere d'acceptation
P0	Suspendre le label legal dans le pipeline ou implementer une vraie chaine dB(A) et residuel trace.	Aucun evenement n'est annonce non conforme sans donnees metrologiques valides; audit de protocole signe.
P0	Installer sounddevice dans la voie de production et ajouter une preflight capture.	Installation propre + service : import, devices et stream configure fonctionnent.
P1	Implementer min_event_hz=2 et freq_max=150 partout.	Configuration, DSP, detection, documentation et tests sont coherents.
P1	Suivre les sessions de nuisance et la validite 10 s.	Duree cumulee et statut INVALIDE sont persistants et exposes par API.
P1	Fermer le serveur par default et documenter le mode d'exposition securise.	127.0.0.1 par default, pas de CORS large, mutation protegee.
P2	Automatiser le nettoyage d'exemplaires apres retention et ajouter observabilite queue/erreurs.	Pas d'orphelin apres retention; compteurs exploitables dans stats/health.
P3	Appliquer ruff format et corriger les caracteres mal encodes.	ruff format --check . passe et sorties CLI lisibles en UTF-8.

Decision de mise en service

Acceptable pour une experimentation de detection technique locale, avec camera sur les dependances audio. Non acceptable en l'etat pour constituer ou filtrer une preuve d'infraction acoustique au sens reglementaire. Apres P0 et P1, realiser une recette sur le HP T620 avec la carte M-Track Plus avant de declarer la version prete au 24/7.

Annexe - fichiers examines

src/bruittrack/{config,legal,dsp,events,pipeline,capture,store,viz,__main__}.py; tests/; pyproject.toml; config.toml.example; systemd/bruittrack.service; tools/{install_hp,module_check,check}.sh/.py; README.md; AGENTS.md; docs/decision-log.md; .github/workflows/ci.yml.

Fin du rapport